



Re-implementation and Security Enhancement of NLP-Based Scam Script Analysis

Stream: Research-Based Project

Course Name: Network Security

Submitted to: Ms. Maryam Sajjad

Submitted By

Class: BESE-13-A

Name	CMS ID
Aimen Munawar	415867
Maheen Akhtar Khan	429419
Hadia Ali	410865

Submission Date: 21-12-2025

Department of Software Engineering
National University of Sciences and Technology (NUST)
Islamabad, Pakistan

Re-implementation and Security Enhancement of NLP-Based Scam Script Analysis

Aimen Munawar, Maheen Akhtar Khan, Hadia Ali
Department of Software Engineering
National University of Sciences and Technology (NUST)
Islamabad, Pakistan

Abstract—This paper shows the re-implementation and security enhancements of the NLP-based crime script analysis methodology proposed by Lwin Tun and Birks (2023). The original paper demonstrated that natural language processing techniques can support crime script analysis by clustering scam reports based on similarities and extracting temporal sequences of scam activities. In this paper, we will reproduce the core NLP pipeline that is presented in the original paper and introduce some improvements, including the utilization of transformer-based embeddings, dual similarity validation, and providing a safer framework for the overall communication of crime reports through various customized techniques for encryption, hashing and digital signatures.

Index Terms—Crime Script Analysis, Scam Detection, Natural Language Processing, Transformer Embeddings, Data Integrity, Information Security, NLP for Security

I. PROBLEM STATEMENT

The increasing number of online scams with more complicated reporting has made manual analysis very slow, prone to errors, and not suitable for large datasets. The current automated methods are not always able to reveal the deeper semantic connections and they also do not pay enough attention to the trustworthiness of the data being analyzed. This leads to the requirement of a better crime script analysis system that would deliver accurate pattern discovery and at the same time provide reliable and trustworthy analysis outcomes.

II. INTRODUCTION

With the increase in the usage of internet, online scams have grown immensely and have become a major cause of financial loss and psychological suffering for victims. In order to design, effective systems for the prevention and detection of such scams, a deeper understanding of the overall working of these scams is required. Crime script analysis solves this issue by providing a method to show these incidents in the form of a series of actions. However, if it is done manually then we would not be able to perform it on larger datasets.

The original paper demonstrates that Natural language processing can be employed for performing crime script analysis automatically. The NLP pipeline achieves this through clustering of scam reports and performing temporal ordering on them. The paper focuses more on the accuracy of the ordering and assumes that the data received as input is trustworthy. In this paper, we aim to make use of more contemporary NLP techniques for the modeling and also focus on ensuring

the confidentiality, integrity, authenticity of the data that the pipeline receives as an input, to avoid misleading crime scripts.

III. ORIGINAL PAPER OVERVIEW

A. Selected Paper

The research paper that we chose is “**Supporting Crime Script Analyses of Scams with Natural Language Processing**” authored by Zeya Lwin Tun and Daniel Birks, which has been published in Crime Science (2023). This paper makes use of NLP methods on scam reports obtained from Singapore’s Scam Alert platform in order to recognize recurrent scams and extract procedural crime scripts from the stories of victims.

B. Justification and Scope of Reproduction

We chose this paper because it matched very well with the areas of cyber crime analysis, NLP application, and information security. The work that we are going to do mainly consists of re-producing the same pre-processing pipeline, similarity computation, and script extraction. We will also be working on semantic modeling and handling security concerns, particularly regarding the safe communication of crime reports, which were not addressed in the original study.

C. Problem Addressed

The original paper addressed the issue of manual creation of crime script from unstructured victim reports, which was time-consuming, susceptible to bias and thus not ideal for large volume of data. The paper solved this issue by using NLP to automatically detect similarities and then perform clustering and temporal ordering, so that we can perform this analysis on a much larger scale.

D. Threat Model

1) Assets Under Protection:

- Raw and pre-processed scam reports data.
- Learned representations including document embeddings and similarity matrices.
- Analytical outputs including scam clusters and crime scripts.

2) Adversary Model:

- An insider with authorized access can modify or inject fake scam reports.
- An outsider can insert manipulated or misleading scam reports.

3) Attack Vectors:

- Scam reports can be tampered with after they are gathered. These altered reports can affect the clustering results.
- Attackers can change the context of valid crime reports.
- These attacks can also involve text paraphrasing to exploit the vulnerabilities of shallow embedding models.

4) Impact of Attacks:

- Similarity scores can be distorted as a result of these attacks.
- Inaccurate scam clusters can be created as output.
- Misleading crime scripts can be generated as a result of this process.
- In short, the overall trust in analysis process will be reduced.

5) Modeling Considerations:

- The original paper utilized Doc2Vec model. This model performs surface-level lexical similarity computation.
- In Doc2Vec model, if an attacker performs textual paraphrasing to insert misleading reports, the model would not provide any protection against that.
- In our re-implementation, we recommend sentence-transformer embedding model, that performs contextual similarity computation.
- Though the integration of transformer model in our pipeline does not remove such threats completely, they do allow us to mitigate their impact on clustering accuracy and crime script generation.

E. System Architecture

The NLP pipeline that is created in the original paper consists of the following phases:

1) *Data Pre-processing*: In this phase, they process raw scam reports by performing the following:

- Removing duplicates, Chinese characters, punctuation marks, stop words.
- Extracting date and informant details
- Generating document ID's
- Validating scam type classification
- Removing very short incidents (length $\geq$ threshold)
- Verifying that each scam category matches original definition
- Expanding acronyms and contractions
- Replacing URL with token
- Correcting misspelled words
- Lemmatizing the tokens

Then, they prepare the corpus by combining the text with document ID's to make it ready for NLP processing.

2) *Embeddings Generation*: In this phase, they trained the Doc2Vec model to learn document representations and then generate embeddings to convert the documents into vectors. They used Average SDQ (Similarity Distance Quality) and Self-similarity index as selection criteria for this model.

3) *Similarity Computation*: In this phase, they compute Cosine and Jaccard similarity to find how similar documents are to each other. Cosine similarity is used to measure similarity between the documents and Jaccard similarity is used to measure the words overlap between the documents. A hybrid approach is also presented that combines both metrics and then uses a top 0.5% threshold.

4) *Clustering*: In this phase, they group together similar scam reports using two methods: threshold-based and graph-based clustering. In threshold-based, documents with similarity greater than threshold form clusters, while in graph-based we create similarity graphs and find components.

5) *Key Terms Extraction*: In this phase, they employ TF-IDF to extract key terms per cluster. They rank terms in cluster based on their TF-IDF score and select the top N terms.

6) *Crime Script Generation*: Now, they will generate crime script and sequence graph by arranging the ranked key terms in sequential order to show typical crime progression.

F. Core Techniques

The main techniques that were used in the paper were Doc2Vec embeddings, cosine similarity, noun-phrase-based Jaccard similarity, TF-IDF term extraction, and median-position-based temporal ordering.

G. Original Evaluation Methodology

The evaluation in the original document was mainly qualitative and was illustrated by a case study on impersonation scams in the High Court. The effectiveness was determined by checking if the produced sequences were in accordance with the scam procedures that were already known.

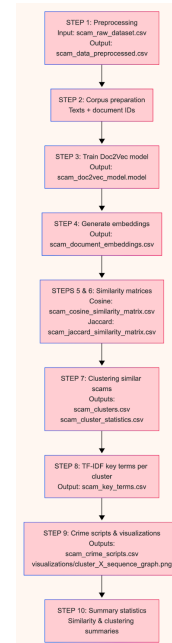


Fig. 1. Original system architecture

IV. RE-IMPLEMENTATION OF ORIGINAL SYSTEM

A. Dataset and Pre-processing

In our implementation of the original paper, we load raw scam reports from CSV files and perform the following steps for pre-processing it:

- Removing URL, punctuations, stop words
- Expanding contraction and acronyms
- Converting to lowercase
- Tokenizing text and lemmatizing words

B. Embedding and Similarity Analysis

In our implementation of the original paper, the system uses sentence-transformers (MiniLM model) as primary method instead of Doc2Vec to generate contextual embeddings. Though we do run Doc2Vec embeddings for comparing this new approach with the original approach.

For finding similarity, the system finds transformer-based cosine similarity to find contextual similarity between documents, finds Jaccard similarity using noun phrases to find measure of words overlap, finds Doc2Vec cosine similarity just for comparison.

C. Clustering

- Our system employs graph-based clustering
- It uses transformer based cosine similarity as main metric (threshold: 0.7) and use Jaccard similarity (threshold: 0.05)
- The system creates clusters by first creating a graph where we create edges between documents if their similarity is greater than similarity thresholds
- It then find connected components in the graphs, which are basically a set of nodes where we can reach any node from any other node via edges
- It then puts all connected components in a single cluster.

D. Crime Script

- Our system ranks key terms in each cluster based on their TF-IDF score and retrieves the top N terms.
- It arranges them based on their median position in documents to find out the usual order of actions in a crime.
- In order to create the crime script, it arranges the key terms in sequential order based on their median position in documents.
- It then visualizes the script using graph, where the nodes are the key terms and the edges show their sequential relationship.

V. IDENTIFIED SECURITY LIMITATIONS

The original assumes that all reports that it receives are trustworthy and does not perform any integrity or authenticity checks on them. In reality, the scam reports could be tampered, injected or poisoned, which can effect our similarity, clustering and crime script. Further, the paper uses Doc2Vec, which finds only surface-level word similarity and thus paraphrased text from attacker can manipulate our results.

VI. PROPOSED SECURITY IMPROVEMENT

In the original system, all the data was saved in plain text CSV files, because of which it was vulnerable to eavesdropping, tampering and other attacks. We enhanced the security of this system by storing the data in secure files that are encrypted, and have digital signatures. We also added an additional layer of security that decrypt all the data, and verifies the integrity and authenticity of the data. Following are the main components of our security improvements:

A. RSA Public Key Cryptography

- We implemented RSA key generation from the scratch, using the Miller–Rabin primality test to produce the prime numbers p and q .
- Our implementation provides adjustable RSA key sizes, where the 256-bit keys are used for demonstration purpose.
- We took advantage of the Chinese Remainder Theorem (CRT) to speed up RSA decryption process.
- We performed RSA encryption with OAEP padding and SHA-256 to securely encrypt small key material like symmetric encryption keys.
- We further added domain-bound labels in each encrypted key, namely `vishing|{crime_type}|{report_id}`, to prevent key-swapping attacks across different reports.
- We appended the AES symmetric key with its initialization vector IV and then encrypted the combined key material using the server's RSA public key.
- OAEP padding with embedded labels was the first step before RSA encryption to guarantee that each encrypted key is cryptographically attached to the corresponding report.
- For digital signatures, we computed the SHA-256 hash over the total report payload which comprised the crime type, report ID, timestamp, and report text.
- We then used the sender's RSA private key to sign this hash and produce a digital signature.

B. AES-256 Symmetric Encryption

- We wrote the functions for AES implementation from scratch that used 256-bit keys for 14 rounds of encryption to process 128-bit input blocks.
- They performed the four fundamental AES operations i.e SubBytes, ShiftRows, MixColumns, and AddRoundKey.
- We used the Cipher block chaining (CBC) mode with PKCS7 padding for encryption, where each plain text block was XORed with the previous cipher text block before encryption and initialization vector (IV) is used for the first block.
- The initialization vector is generated from the string `vishing_iv|{crime_type}|{report_id}|{timestamp}`, which is then hashed using SHA-256, and the first 16 bytes of the hash are used as the initialization vector.

C. SHA-256 Hash Function

- We implemented the SHA-256 hash function from scratch that took 512-bit chunks as input and produced their 256-bit hashes.
- In order to make sure that the hashes are unique, even if the content of the reports are identical, we added crime type, report ID, timestamp, and canonicalized text in the payload of hashes.

D. HMAC-SHA256 Message Authentication

- We implemented it by first generating an inner hash with the inner key formed by the XORing of the HMAC key with the ipad constant and hashing it together with the message using SHA-256. After that we found the outer hash with the outer key formed with the opad constant.
- This resulted in the generation of HMAC hash that helps us in verifying if there has been any tampering in our data.
- To find the HMAC key, we combined the AES keys with report metadata like crime type, report ID, and timestamp.

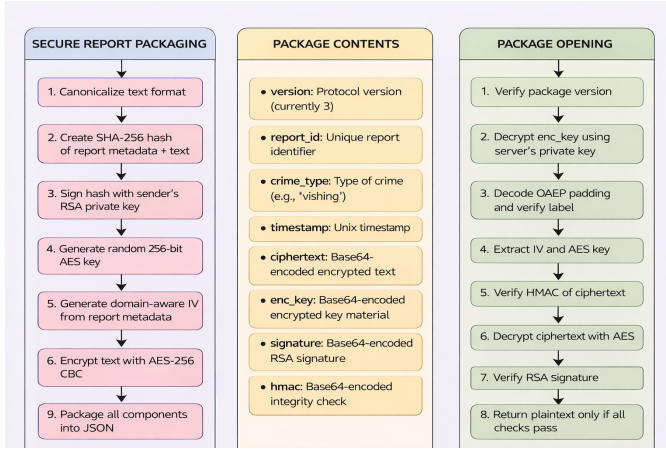


Fig. 2. Security Improvements

VII. ATTACK AND SECURITY ANALYSIS

A. Attack Model

In our system, we have analyzed the following seven attack scenarios:

- **Eavesdropping:** We try to intercept network traffic to read the reports.
- **Tampering:** We try to modify the report content during storage or transmission.
- **Replay:** We try to capture and retransmit old reports.
- **Man-in-the-Middle:** We try to intercept communication and create fake reports.
- **Signature Forgery:** We try to create valid signatures without authorization.
- **Key Swapping:** We try to swap encryption keys between different reports.
- **Pattern Analysis:** We try to detect patterns in encrypted data to identify similar reports.

B. Effect on Original System

- **Eavesdropping:** As the input data was plain text CSV files, attacker could read all content directly.
- **Tampering:** As the system had no integrity checks, any modifications were undetected.
- **Replay:** In the original system there were no timestamps, so old reports can be resubmitted.
- **Man-in-the-Middle:** As there was no authentication, so fake reports were accepted easily.
- **Signature Forgery:** There was no signature mechanism in this system.
- **Key Swapping:** As plain text data was directly processed, so the system was vulnerable to this attack too. Pattern Analysis: The plain text storage of data revealed all the patterns.

C. Effect on Improved System

- **Eavesdropping:** In our system we prevented it through AES-256 encryption, so attacker sees only encrypted data.
- **Tampering:** It was prevented by HMAC-SHA256 and RSA signatures that help in detecting modifications
- **Replay:** The Timestamps in hash and IV generation prevented it.
- **Man-in-the-Middle:** Through RSA signatures sender identity can be proved, and thus this attack can be prevented.
- **Signature Forgery:** This attack was prevented as our system requires sender's private key, further the hash also includes report ID and timestamp.
- **Key Swapping:** The OAEP labels that bind keys to specific reports helped in blocking this attack.
- **Pattern Analysis:** Using the unique IVs per report, we make sure that same text produces different cipher texts.

TABLE I
SECURITY COMPARISON BETWEEN ORIGINAL AND ENHANCED SYSTEM

Security Property	Original System	Enhanced System
Confidentiality	Plaintext	AES-256 + RSA-OAEP
Integrity	None	HMAC-SHA256
Authentication	None	RSA Signatures
Non-repudiation	None	Digital Signatures
Replay Protection	None	Timestamp Binding
Tamper Detection	None	HMAC + Signatures
Pattern Analysis	Vulnerable	Unique IVs
Eavesdropping Resistance	Vulnerable	Full Encryption

VIII. EVALUATION

The original system utilized the Doc2Vec model to compute Cosine and Jaccard similarity between scam reports. In our re-implementation, we replace Doc2Vec with a sentence transformer model while retaining Cosine and Jaccard similarity measures.

A. Similarity Analysis

The findings show that transformer embeddings have a greater similarity scores, revealing their power in capturing the semantics of scam reports. On the other hand, Jaccard similarity is still low since it is based on no more than precise word overlap.

TABLE II
EMBEDDING METHOD COMPARISON (SIMILARITY STATISTICS)

Method	Mean	Std	Min	Max
Transformer	0.4802	0.1367	-0.0443	1.0000
Doc2Vec	0.2664	0.1991	-0.4851	1.0000
Jaccard	0.0277	0.0717	0.0000	1.0000

TABLE III
CORRELATION BETWEEN SIMILARITY METHODS

Method Pair	Pearson Correlation
Transformer – Doc2Vec	0.3507
Transformer – Jaccard	0.5714
Doc2Vec – Jaccard	0.4286

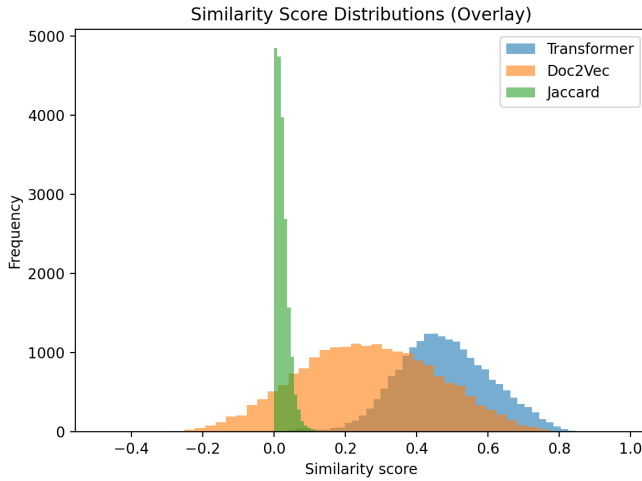


Fig. 3. Overlay histogram comparing similarity score distributions for Transformer, Doc2Vec, and Jaccard methods.

B. Clustering Analysis

The findings reveal that a great number of scam reports have similar characteristics, thus resulting in the formation of bigger clusters that signify the recurrence of certain scam methods. Figure 4 shows the distribution of cluster sizes obtained from

TABLE IV
CLUSTERING RESULTS SUMMARY

Metric	Value
Total clusters	87
Average cluster size	46.93
Largest cluster	96 documents
Clusters with key terms	11
Clusters with crime scripts	11

scam report clustering.

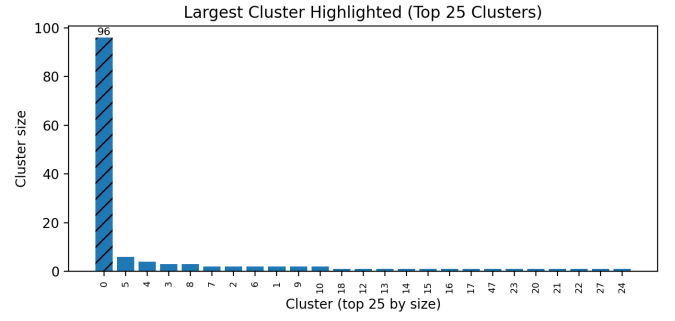


Fig. 4. Top 25 cluster size distribution obtained from scam report clustering.

C. Case Study

To evaluate the system on an unseen example, we analyzed a new scam report:

“I received a scam call. It was an automated voice from the Singapore High Court, stating that I have an outstanding summon. I was asked to pay it.”

TABLE V
EXTRACTED CRIME SCRIPT STEPS FOR THE CASE STUDY

Step	Action	Importance	Next Term
1	local	0.1562	call
2	call	0.3123	number
3	number	0.1562	answer
4	answer	0.1562	automate
5	automate	0.1562	voice
6	voice	0.1562	english
7	english	0.1562	singapore
8	singapore	0.4685	high
9	high	0.3123	immediately
10	immediately	0.1562	court
11	court	0.3123	hang
12	hang	0.1562	total
13	total	0.1562	serve
14	serve	0.3123	scam
15	scam	0.1562	summon
16	summon	0.3123	never
17	never	0.1562	like
18	like	0.1562	—

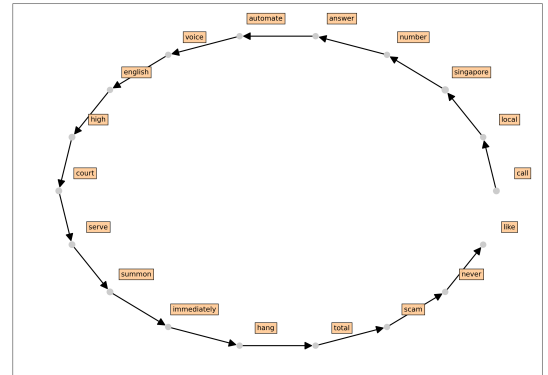


Fig. 5. Crime script sequence graph generated for the case study report.

IX. MAPPING TO SECURITY OBJECTIVES

A. Confidentiality

Our system ensures data confidentiality through AES-256 encryption for report text and RSA-OAEP encryption for keys. Further, each report uses unique IV's, so identical reports produce different cipher texts. Only the authorized user can decrypt this data with their private key.

B. Integrity

Our system uses two techniques to prevent the tampering of data ie. HMAC-SHA256 that verifies encrypted data integrity, and RSA digital signatures to ensure authenticity of reports. If any of the verifications fail the report gets rejected without further processing.

C. Authentication

In our system, RSA signature proves the identity of sender. As, only the sender with private key can generate valid signatures. Further, the signatures ensure non-repudiation as the sender cannot deny sending the message.

X. LIMITATIONS AND FUTURE WORK

A. Limitations

Our implementation uses 256-bit RSA keys which are weaker than the standard production standard of 2048-bit. Further, we have implemented all the cryptographic algorithms from scratch, so they can have some vulnerabilities. Our system also does not provide any key rotation or management policies.

B. Future Work

Our future improvements will include proper key management with rotation and backup, performance optimization for larger datasets, additional attack testing (side-channel, timing attacks), and TLS integration for network security.

XI. CONCLUSION

This paper shows our re-implementation of NLP-based crime script analysis system with comprehensive security enhancements. Where in place of Doc2Vec model, sentence-transformer embeddings were used and cryptographic techniques (RSA, AES-256, SHA-256, HMAC) were implemented to scam reports confidentiality, integrity, and authentication.

REFERENCES

- [1] Z. L. Tun and D. Birks, "Supporting crime script analyses of scams with natural language processing," *Crime Science*, vol. 12, no. 1, Feb. 2023, doi: 10.1186/s40163-022-00177-w.
- [2] J. Jonsson and B. Kaliski, "Public-Key Cryptography Standards (PKCS) #1: RSA Cryptography Specifications Version 2.1," RFC 3447, Feb. 2003, doi: 10.17487/RFC3447.
- [3] National Institute of Standards and Technology, "Advanced Encryption Standard (AES)," FIPS 197, May 2023, doi: 10.6028/NIST.FIPS.197-upd1.
- [4] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS 180-4, Jan. 2015, doi: 10.6028/NIST.FIPS.180-4.
- [5] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," RFC 2104, Feb. 1997. [Online].
- [6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. 2019 Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP)*, Hong Kong, China, 2019, pp. 3982–3992.
- [7] W. Wang, K. Wei, F. Deng, D. Chen, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Proc. 34th Int. Conf. Neural Information Processing Systems (NeurIPS)*, 2020.